

Deep Reinforcement Learning for Software Quality Assurance

Test Case Prioritization with DQN and PPO

Datasets: NASA KC1 & JM1

MSc Artificial Intelligence — Deep Learning & Multimedia Data Analysis

1. Introduction

This report presents a Deep Reinforcement Learning (DRL) approach to the software quality assurance problem of Test Case Prioritization (TCP). The goal is to train an RL agent that learns to identify defect-prone software modules based on static code metrics, enabling a QA engineer to allocate testing resources more efficiently — finding more defects while running fewer tests.

Two deep RL algorithms are compared on two NASA software defect datasets:

DQN (Deep Q-Network): Learns a Q-function $Q(\text{state}, \text{action})$ that estimates the expected cumulative reward for testing or skipping each module. Uses experience replay and a target network for stable training.

PPO (Proximal Policy Optimization): Learns a policy $\pi(\text{action}|\text{state})$ directly, mapping module metrics to action probabilities. Clips policy updates to avoid destructive large gradient steps, making training more stable.

Both algorithms are evaluated on the NASA KC1 and JM1 datasets from the PROMISE Software Engineering Repository, enabling a cross-dataset analysis of how each algorithm adapts to different defect rates and dataset sizes.

The source code and experimental results are available at the following links:

- [GitHub Repository](#)
- [KC1 Experiment](#)
- [JM1 Experiment](#)

2. Methodology

2.1 Problem Formulation

Test Case Prioritization is formulated as a Markov Decision Process (MDP). At each step, the agent observes the software metrics of one module and decides whether to test it. The episode ends when all modules have been reviewed exactly once.

State: A vector of 21 normalized software complexity metrics for the current module (Halstead metrics + McCabe cyclomatic complexity).

Action: Discrete(2) — Action 0 = SKIP the module, Action 1 = TEST the module.

Reward: +9 for finding a defect (TEST a defective module), -1 for a wasted test (TEST a clean module), 0 for correctly skipping a clean module, -20 for missing a defect (SKIP a defective module). The -20 penalty reflects the real-world cost of a defect escaping to production.

Episode: One full pass through all test-set modules. The agent's objective is to maximize total episode reward.

2.2 Datasets

Both datasets are from the NASA PROMISE Software Engineering Repository. Each row represents one software module with 21 Halstead and McCabe metrics as features and a binary defect label as the target. Table 1 summarizes the key properties.

Property	KC1	JM1	Difference	Impact
Source system	Storage mgmt (C++)	Real-time prediction (C)	—	—
Total modules	2,109	10,878	5× larger	More training data
Defect rate	15.5%	19.4%	+3.9%	Denser signals
Training modules	1,687 (268 defective)	8,702 (1,679 defective)	—	—
Test modules	422 (67 defective)	2,176 (420 defective)	—	—
Features	21	21	Identical	Fair comparison

Table 1. Dataset properties for KC1 and JM1. Both share the same 21 features but differ in size and defect rate.

The 21 features include lines of code (loc), cyclomatic complexity (v_g), Halstead volume (v), Halstead difficulty (d), Halstead effort (e), branch count, and related metrics. All features are normalized to [0, 1] using MinMaxScaler fitted on the training set only, preventing data leakage. The datasets are split 80/20 (train/test), stratified to preserve defect rates.

2.3 Custom Gymnasium Environment

A custom environment (SoftwareQAE_{env}) was implemented following the Gymnasium API. Modules are shuffled at the start of each training episode to prevent the agent from memorizing orders. The training set is used during learning; the test set is held out and used only for final evaluation.

2.4 Algorithm Configuration

Both algorithms use an MLP policy with two hidden layers of 64 neurons each. The same architecture is used for both to ensure a fair comparison. Training runs for 200,000 timesteps on each dataset.

Setting	DQN — KC1	PPO — KC1	DQN — JM1	PPO — JM1
Total timesteps	200,000	200,000	200,000	200,000
Training time	3.8 min	7.4 min	3.8 min	7.0 min
Optimizer	Adam	Adam	Adam	Adam
Network	MLP [64, 64]	MLP [64, 64]	MLP [64, 64]	MLP [64, 64]
Eval episodes	50	50	50	50

Table 2. Training configuration. Both algorithms share the same network architecture and training budget.

DQN configuration: learning rate 1e-3, replay buffer 50,000, batch size 64, gamma 0.95, target network update every 500 steps, epsilon decay from 1.0 to 0.05 over 30% of training.

PPO configuration: learning rate 3e-4, n_steps 512, batch size 64, 10 optimization epochs per rollout, gamma 0.95, GAE lambda 0.95, clip range 0.2, entropy coefficient 0.01.

2.5 Baseline

A random agent that selects TEST or SKIP with equal probability (50/50) serves as the performance baseline. This represents the expected behavior of a QA process with no knowledge of which modules are more likely to be defective. Both DQN and PPO must outperform this baseline to demonstrate genuine learning.

3. Results

3.1 KC1 Dataset

Table 3 summarizes results on the KC1 test set (422 modules, 67 defective) averaged over 50 evaluation episodes.

Agent	Recall	Test Rate	Precision	Total Reward	
Random Baseline	51.0%	50.1%	16.2%	-525	—
DQN	68.7%	39.1%	27.9%	-125	↑
PPO	85.1%	55.0%	24.6%	+138	↑↑

Table 3. KC1 results (422 test modules, 67 defective). Green = best value per metric.

3.2 JM1 Dataset

Table 4 summarizes results on the JM1 test set (2,176 modules, 420 defective) averaged over 50 evaluation episodes.

Agent	Recall	Test Rate	Precision	Total Reward	
Random Baseline	50.0%	50.0%	19.3%	-3,193	—
DQN	62.6%	21.6%	56.1%	-979	↑
PPO	99.5%	96.2%	20.0%	+2,046	↑↑

Table 4. JM1 results (2,176 test modules, 420 defective). Green = best value per metric.

3.3 Cross-Dataset Comparison

Table 5 places all results side by side for direct cross-dataset and cross-algorithm comparison.

Agent	KC1 Recall	KC1 Test Rate	KC1 Reward	JM1 Recall	JM1 Test Rate	JM1 Reward
Random	51.0%	50.1%	-525	50.0%	50.0%	-3,193
DQN	68.7%	39.1%	-125	62.6%	21.6%	-979
PPO	85.1%	55.0%	+138	99.5%	96.2%	+2,046

Table 5. Complete cross-dataset comparison. Green = best value per metric column.

3.4 Learning Curves

The learning curves below show mean episode reward during training for both algorithms on both datasets.

DQN vs PPO – KC1 Dataset

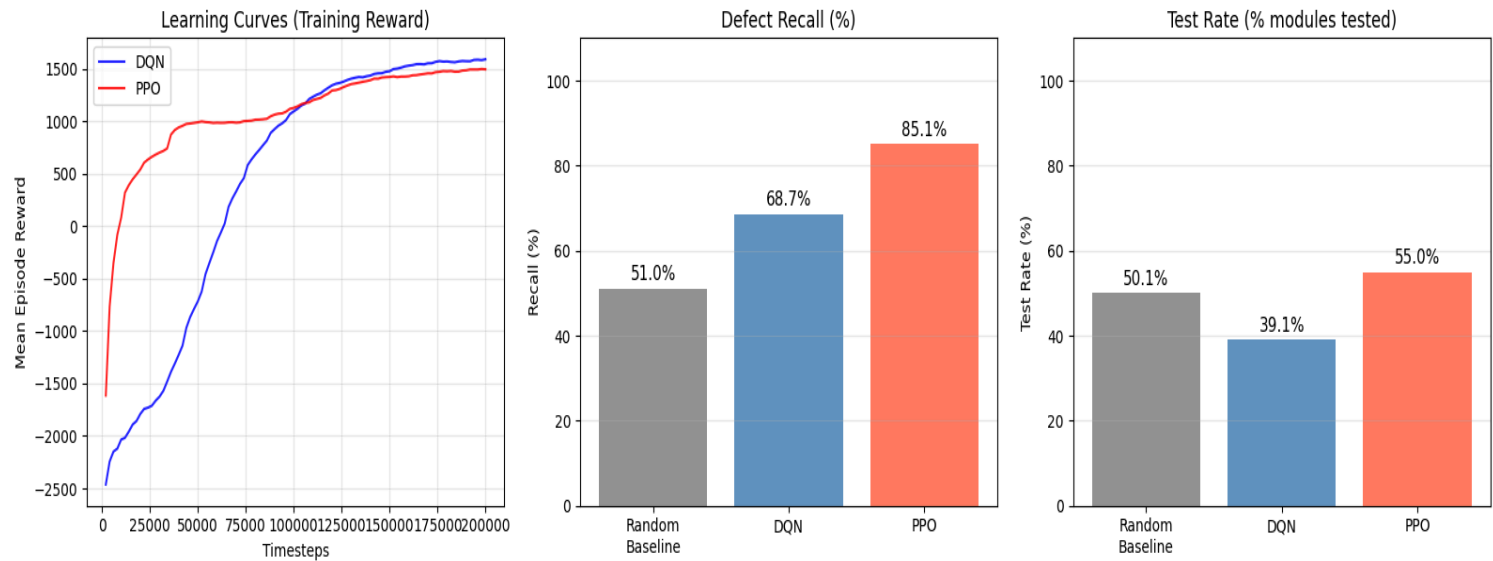


Figure 1. KC1 results — learning curves, defect recall, and test rate comparison for DQN and PPO. PPO achieves higher recall; DQN achieves lower test rate.

DQN vs PPO – JM1 Dataset

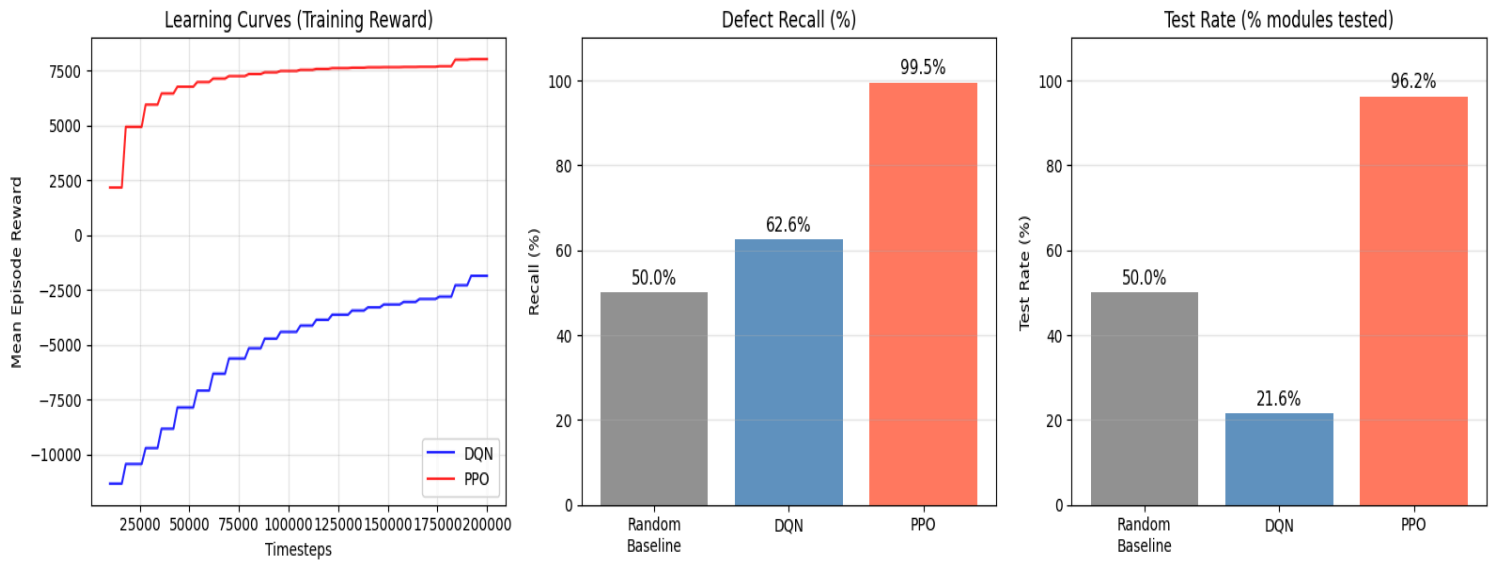


Figure 2. JM1 results. The strategy divergence between DQN and PPO is amplified on the larger dataset: DQN tests only 21.6% of modules while PPO tests 96.2%.

4. Findings & Analysis

4.1 Both Agents Outperform Random Testing

The random baseline achieves 51.0% recall on KC1 and 50.0% on JM1 — as expected from a 50/50 random policy. Both DQN and PPO improve substantially over this baseline on recall and total reward, confirming that the agents learned to exploit the statistical relationship between software metrics and defect presence. This validates the core premise: complexity metrics carry predictive signals that an RL agent can learn to use.

4.2 DQN vs PPO — Divergent Learned Strategies

The most significant finding is that DQN and PPO converge to fundamentally different strategies despite being trained in the same environment with the same reward function.

DQN adopts a precision-focused strategy: it is highly selective, testing only modules where it has high confidence of a defect. On JM1, DQN tests just 21.6% of modules and achieves 56.1% precision — meaning over half of modules it selects actually contain defects, three times better than random (19.3%). The trade-off is lower recall (62.6%): DQN misses approximately one third of defects.

PPO adopts a recall-focused strategy: it tests broadly, prioritizing coverage to avoid the -20 missed-defect penalty. On JM1, PPO tests 96.2% of modules and achieves 99.5% recall — missing only 2 out of 420 defects. The trade-off is low precision (20.0%) and near-random test selection efficiency.

This strategic divergence is explained by the fundamental algorithmic difference: DQN optimizes Q-values independently per module and bets on high-confidence modules. PPO optimizes a sequential policy with entropy regularization, which encourages broad exploration and pushes toward high coverage when the missed-defect penalty is large.

4.3 Dataset Size Amplifies Strategy Divergence

On KC1 (2,109 modules), the recall gap between DQN and PPO is 16.4 percentage points (68.7% vs 85.1%) and the test rate gap is 15.9 points (39.1% vs 55.0%). On JM1 (10,878 modules), the recall gap widens to 36.9 points (62.6% vs 99.5%) and the test rate gap to 74.6 points (21.6% vs 96.2%). Larger datasets with richer patterns allow DQN to learn a more extreme selective strategy, while PPO's entropy bonus pushes it toward near-complete coverage.

4.4 PPO Achieves Higher Total Reward on Both Datasets

PPO achieves the highest total reward on both datasets (+138 vs -125 on KC1, +2,046 vs -979 on JM1), as the reward function penalizes missed defects (-20) far more than wasted tests (-1), making recall the dominant factor. DQN's negative reward reflects missed defect costs rather than poor performance — it still finds significantly more defects per test executed than the random baseline.

4.5 Practical Recommendation

The choice between DQN and PPO should be guided by the QA objective rather than a single metric. PPO is recommended for safety-critical systems where missing a defect is unacceptable and test execution time is not a constraint. DQN is recommended for resource-constrained pipelines where testing capacity is limited and high precision is valued over complete coverage. This complementarity makes the two algorithms tools for different deployment contexts rather than competitors for a single universal role.

4.6 Training Efficiency

Both algorithms complete training in under 8 minutes per dataset on CPU, making the approach practical for real QA workflows that need to retrain periodically as codebases evolve. PPO is approximately 1.9× slower than DQN (7.4 min vs 3.8 min on KC1) due to its on-policy nature requiring more computation per update. The larger JM1 dataset does not significantly increase training time (3.8 min for DQN, 7.0 min for PPO) because the number of timesteps is fixed at 200,000 regardless of dataset size.

5. Conclusions

This assignment applied Deep Reinforcement Learning to software test case prioritization, training DQN and PPO agents to identify defect-prone modules from static complexity metrics on two NASA datasets. The key conclusions are:

- Both DQN and PPO significantly outperform random testing on both datasets, validating that software complexity metrics carry a learnable signal for defect prediction.
- DQN and PPO converge to complementary strategies: DQN prioritizes precision (fewer tests, higher hit rate) while PPO prioritizes recall (broader coverage, fewer missed defects). Neither strategy is universally superior — the choice depends on the QA objective.
- Dataset size amplifies strategy divergence: on the larger JM1 dataset, DQN tests only 21.6% of modules (vs 39.1% on KC1) while PPO approaches near-complete coverage (96.2% vs 55.0%), demonstrating that richer data allows DQN to learn more extreme selective behavior.
- PPO achieves the highest total reward on both datasets, driven by its recall-focused strategy aligned with the reward function's large penalty for missed defects (+2,046 on JM1 vs -979 for DQN).
- Training is fast (under 8 minutes per dataset per algorithm on CPU), making the approach viable for periodic retraining as codebases evolve.

Key Takeaways

Across all three assignments, a consistent theme emerges: the choice of algorithm and architecture matters more than the specific configuration. In Assignment 1, switching from a poor configuration (ResNet18 FE on CIFAR-10) to a better one (ResNet18 FT) required significant fine-tuning effort. In Assignment 2, switching architectures entirely (ResNet18 to ViT) proved more impactful than fine-tuning strategy. In this assignment, algorithm choice (DQN vs PPO) determines not just performance magnitude but the fundamental nature of the learned strategy — a precision tool versus a coverage tool. This suggests that algorithm selection should be driven by domain requirements rather than default choices.